Πανεπιστήμιο Δυτικής Αττικής
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Εργαστηριακή Άσκηση 1

ΓΙΑ ΤΟ ΜΑΘΗΜΑ

«ΟΡΑΣΗ ΥΠΟΛΟΓΙΣΤΩΝ»

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: ΙΩΑΝΝΗΣ ΤΕΡΠΟΣ

ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ: 18390208

ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ: ΠΑΔΑ

ΕΡΓΑΣΤΗΡΙΑΚΗ ΟΜΑΔΑ Α: ΠΑΡΑΣΚΕΥΗ 15:05 - 16:30

ΑΙΘΟΥΣΑ: ΕΡΓΑΣΤΗΡΙΟ ΤΕΧΝΗΤΗΣ ΝΟΗΜΟΣΥΝΗΣ ΚΑΙ ΠΟΛΥΜΕΣΩΝ(Κ10.022)

ΧΕΙΜΕΡΙΝΟ ΕΞΑΜΗΝΟ

Περιεχόμενα

1. Εισαγωγή 3
2. Bag of Visual Words image classification 3
2.1. Feature Extraction 3
2.1.1. SIFT 3
2.1.2. BRISK 3
2.1.3. BRIEF 4
2.2. Κατασκευή λεξικού 5
2.2.1 K-means 5
2.2.2. Meanshift 5
2.3. Παρουσίαση και σχολιασμός αποτελεσμάτων 5
3. Semantic Segmentation 6
3.1. Features 7
3.1.1. Χρώμα 7
3.1.2. Ακμές 7
3.1.3. Εντροπία 7
3.1.4. Εξισορρόπηση 7
3.1.5. Γωνίες 7
3.2. Παρουσίαση και σχολιασμός αποτελεσμάτων 8
3.3. Γραφικές Παραστάσεις predicted και original annotated masks 8
4. Συμπεράσματα 15

1. Εισαγωγή

Η εργαστηριακή άσκηση 1 στοχεύει στην χρήση αλγορίθμων εξαγωγής συγκεκριμένων χαρακτηριστικών (Feature extraction), στη ταξινόμηση εικόνων (Image Classification) χρησιμοποιώντας διάφορους ταξινομητές (classifiers) και στη σημασιολογική κατάτμηση (Semantic Segmentation). Στην ενότητα 2, θα αναφερθούμε στη χρήση της τεχνικής Bag of Visual Words για την ταξινόμηση ενός dataset, που θα αποτελείται από τρεις διαφορετικές κατηγορίες εικόνων. Στην ενότητα 3, θα επισημανθεί το Scikit-learn για τη σημασιολογική κατάτμηση εικόνων. Στο τέλος κάθε ενότητας, θα υπάρχει παρουσίαση και σχολιασμός αποτελεσμάτων για τις τεχνικές που θα χρησιμοποιηθούν.

2. Bag of Visual Words image classification

Σε αυτή την ενότητα, θα χρησιμοποιηθεί ο κώδικας **BoVWClassification.py**, στον οποίον θα επεξηγηθούν διάφοροι αλγόριθμοι εξαγωγής συγκεκριμένων χαρακτηριστικών και κατασκευής λεξικού. Τα links του dataset, που χρησιμοποιούνται για αυτό το κώδικα, βρίσκονται στο φάκελο Dataset και, πιο συγκεκριμένα, στο αρχείο **Σύνδεσμοι_ερωτήματος1.txt** του φακέλου Ερώτημα1. Επίσης , στο αρχείο **Σύνδεσμοι_ερωτήματος1.txt**, αναφέρονται ποιοι σύνδεσμοι του dataset χρησιμοποιούνται στο train data και ποιοι στο test data για κάθε αναλογία train data/ test data.

2.1. Feature Extraction

Οι τεχνικές περιγραφής σημείων ενδιαφέροντος που χρησιμοποιούνται στο κώδικα **BoVWClassification.py** είναι ο **SIFT**, ο **BRISK** και ο **BRIEF**. Στις παρακάτω υποενότητες, αναφέρεται η λειτουργία κάθε αλγορίθμου.

2.1.1. SIFT

Ο SIFT εξάγει χαρακτηριστικά γνωρίσματα, τα οποία χρησιμοποιούνται για την αντιστοίχιση αντικειμένων σε διαφορετικές εικόνες. Για κάθε σημείο ενδιαφέροντος, ένας feature descriptor υπολογίζεται από το μέγεθος της κλίσης και τον σχετικό προσανατολισμό στην τοπική γειτονιά των εικονοστοιχείων. Παρακάτω, έχουμε τη συνάρτηση στο κώδικα **BoVWClassification.py** που υλοποιεί το παραπάνω αλγόριθμο.

```
def detector_features_SIFT(images):
   print(' . start detecting points and calculating features for a given image set with algorithm SIFT')
   detector_vectors = {}
   descriptor_list = []
   detectorToUse = cv2.xfeatures2d.SIFT_create() #initialize the SIFT detector algorithm
   for nameOfCategory, availableImages in images.items():
      print(' . we are in category:', nameOfCategory )
      features = [] #Initialize the list
      tmpImgCounter = 1
      for img in availableImages:
         kp, des = detectorToUse.detectAndCompute(img, None) # compute the descriptors and keypoints with SIFT
         tmpImgCounter += 1
         if des is None:
            print(' .. WARNING: image {:d} cannot be used'.format(tmpImgCounter))
         else:
            descriptor_list.extend(des)
            features.append(des)
      detector_vectors[nameOfCategory] = features
      print(' . finished detecting points and calculating features for a given image set with algorithm SIFT')
   return [descriptor_list, detector_vectors]
```

2.1.2. BRISK

Στον αλγόριθμο BRISK, αφού επιλεγούν τα σημεία ενδιαφέροντος, εφαρμόζεται ένα πρότυπο δειγματοληψίας στη γειτονιά του σημείου

ενδιαφέροντος. Στη συνέχεια, τα ζεύγη εικονοστοιχείων γύρω από το σημείο ενδιαφέροντος χωρίζονται σε δύο υποσύνολα: ζεύγη μικρών και μεγάλων αποστάσεων. Τα ζεύγη μεγάλων αποστάσεων χρησιμοποιούνται για τον καθορισμό του προσανατολισμού και τα ζεύγη μικρών αποστάσεων χρησιμοποιούνται για τις συγκρίσεις έντασης με αποτέλεσμα να κατασκευαστεί ο descriptor. Παρακάτω, έχουμε τη συνάρτηση στο κώδικα **BoVWClassification.py** που υλοποιεί το παραπάνω αλγόριθμο.

```
def detector_features_BRISK(images):
    print(' . start detecting points and calculating features for a given image set with algorithm BRISK')
    detector_vectors = {}
    descriptor_list = [] #Initialize the list
    detectorToUse = cv2.BRISK_create() #initialize the BRISK detector algorithm
    for nameOfCategory, availableImages in images.items():
        print(' . we are in category:', nameOfCategory )
        features = []
        tmpImgCounter = 1
        for img in availableImages:
            kp, des = detectorToUse.detectAndCompute(img, None) # compute the descriptors and keypoints with BRISK
            tmpImgCounter += 1
            if des is None:
                print(' .. WARNING: image {:d} cannot be used'.format(tmpImgCounter))
            else:
                descriptor_list.extend(des)
                features.append(des)
        detector_vectors[nameOfCategory] = features
        print(' . finished detecting points and calculating features for a given image set with algorithm BRISK')
    return [descriptor_list, detector_vectors]
```

2.1.3. BRIEF

Ο BRIEF descriptor βασίζεται σε έναν σχετικά μικρό αριθμό δοκιμών διαφοράς έντασης για να αναπαραστήσει μια ενημέρωση τμήματος εικόνας ως δυαδική συμβολοσειρά. Όχι μόνο είναι αποδοτικά γρήγορος από άλλες τεχνικές σημείων ενδιαφέροντος, αλλά τείνει και σε υψηλότερα ποσοστά αναγνώρισης. Παρακάτω, έχουμε τη συνάρτηση στο κώδικα **BoVWClassification.py** που υλοποιεί το παραπάνω αλγόριθμο.

```
def detector_features_BRIEF(images):
    print(' . start detecting points and calculating features for a given image set with algorithm BRIEF')
    detector_vectors = {}
    descriptor_list = []
    fast = cv2.xfeatures2d.StarDetector_create() # initialize FAST detector
    brief = cv2.xfeatures2d.BriefDescriptorExtractor_create() # Initiate BRIEF extractor
    for nameOfCategory, availableImages in images.items():
        print(' . we are in category:', nameOfCategory )
        features = [] #Initialize the list
        tmpImgCounter = 1
        for img in availableImages:
            # find the keypoints with STAR
            kp = fast.detect(img, None)

            # compute the descriptors with BRIEF
            kp, des = brief.compute(img, kp)
            tmpImgCounter += 1
            descriptor_list.extend(des)
            features.append(des)

        detector_vectors[nameOfCategory] = features
        print(' . finished detecting points and calculating features for a given image
```

```
set with algorithm BRIEF,')
    return [descriptor_list, detector_vectors]
```

2.2. Κατασκευή λεξικού

Στο κώδικα **BoVWClassification.py**, χρησιμοποιούνται δύο αλγόριθμοι συσταδοποίησης (clustering algorithms) για τη κατασκευή του λεξικού. Αυτοί οι αλγόριθμοι είναι ο k-means και ο Meanshift. Στις παρακάτω υποενότητες, αναλύονται οι λειτουργίες αυτών των αλγορίθμων.

2.2.1 K-means

Ο k-means είναι ένας επαναληπτικός αλγόριθμος συσταδοποίησης που χωρίζει ένα πλήθος δεδομένων σε k συστάδες με βάση τα κριτήρια ομοιότητας και τη μέση απόσταση τους από το κέντρο της συγκεκριμένης συστάδας. Παρακάτω, έχουμε τη συνάρτηση στο κώδικα **BoVWClassification.py** που υλοποιεί το παραπάνω αλγόριθμο.

```
def kmeansVisualWordsCreation(k, descriptor_list):
    print(' . calculating central points for the existing feature values.')
    batchSize = np.ceil(descriptor_list.__len__()/50).astype('int')
    kmeansModel = MiniBatchKMeans(n_clusters=k, batch_size=batchSize,
verbose=0)
 # Use k-means algorithm to create the model
    kmeansModel.fit(descriptor_list)  # train the model
    visualWords = kmeansModel.cluster_centers_ # centers of reference
    print(' . done calculating central points for the given feature set.')
    return visualWords, kmeansModel # Return the Bag of Visual Words and the
K-Means model.
```

2.2.2. Meanshift

Σε αντίθεση με το k-means, ο Meanshift είναι ένας ιεραρχικός αλγόριθμος συσταδοποίησης, στον οποίον δε χρειάζεται να γνωρίζουμε τον αριθμό των συστάδων. Βρίσκει τα μέγιστα των στοιχείων και χρησιμοποιείται για να λαμβάνουμε υπόψη τη σύγκλιση στη συνάρτηση πυκνότητας. Παρακάτω, έχουμε τη συνάρτηση Meanshift στο κώδικα **BoVWClassification.py**.

```
def MeanShiftVisualWordsCreation(descriptor_list):
    print('Calculating central points for the existing feature values with Mean
Shift.')
    ShiftModel = MeanShift(bandwidth=10)  # Use MeanShift to create the model.
    ShiftModel.fit(descriptor_list) # train the model
    VisualWords = ShiftModel.labels_  # centers of reference.
    print('Done calculating central points for the given feature set with Mean
Shift.')
    return VisualWords, ShiftModel  # Return the Bag of Visual Words and the
Mean Shift model.
```

2.3. Παρουσίαση και σχολιασμός αποτελεσμάτων

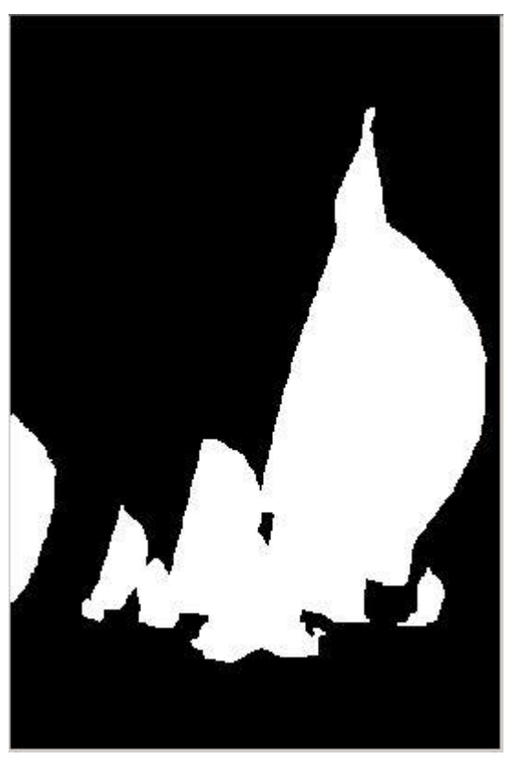
Στο φύλλο **BOVWImageClassification** του αρχείου **results.xlsx**, έχουμε τα αποτελέσματα με αναλογίες train data/test data 80%/20% και 60%/40% για κάθε αλγόριθμο εξαγωγής συγκεκριμένων χαρακτηριστικών και συσταδοποίησης. Για τον υπολογισμό των αποτελεσμάτων, χρησιμοποιούνται μηχανές διανύσματος υποστήριξης (support vector machines).

FeatureExtraction	Clustering Detection	Train/Test Data ratio	Classifier Used	Accuracy (tr)	Precision (tr)	Recall (tr)	F1 score (tr)	Accuracy (te)	Precision (te)	Recall (te)	F1 score (te)
SIFT	Kmeans	80%/20%	Support Vector Machines	0.88	0.93	0.83	0.85	0.75	0.83	0.83	0.78
SIFT	MeanShift	80%/20%	Support Vector Machines	1.00	1.00	1.00	1.00	0.25	0.08	0.33	0.13
BRISK	Kmeans	80%/20%	Support Vector Machines	0.94	0.96	0.92	0.93	0.50	0.44	0.50	0.39
BRISK	MeanShift	80%/20%	Support Vector Machines	1.00	1.00	1.00	1.00	0.25	0.08	0.33	0.13
BRIEF	Kmeans	80%/20%	Support Vector Machines	0.88	0.93	0.85	0.87	0.25	0.11	0.33	0.17
BRIEF	MeanShift	80%/20%	Support Vector Machines	1.00	1.00	1.00	1.00	0.25	0.08	0.33	0.13
SIFT	Kmeans	60%/40%	Support Vector Machines	0.83	0.90	0.78	0.78	0.38	0.24	0.39	0.30
SIFT	MeanShift	60%/40%	Support Vector Machines	1.00	1.00	1.00	1.00	0.38	0.12	0.33	0.18
BRISK	Kmeans	60%/40%	Support Vector Machines	1.00	1.00	1.00	1.00	0.38	0.53	0.39	0.40
BRISK	MeanShift	60%/40%	Support Vector Machines	1.00	1.00	1.00	1.00	0.38	0.12	0.33	0.18
BRIEF	Kmeans	60%/40%	Support Vector Machines	0.83	0.88	0.78	0.77	0.38	0.14	0.33	0.20
BRIEF	MeanShift	60%/40%	Support Vector Machines	1.00	1.00	1.00	1.00	0.38	0.12	0.33	0.18

Παρατηρούμε ότι οι αλγόριθμοι BRISK και k-means δίνουν τα καλύτερα αποτελέσματα για αναλογία train data/test data 60%/40%, ενώ ο αλγόριθμος Meanshift και όλες οι τεχνικές περιγραφής σημείων ενδιαφέροντος δίνουν τα καλύτερα αποτελέσματα με αναλογία train data/test data 80%/20%.

3. Semantic Segmentation

Σε αυτή την ενότητα, θα χρησιμοποιηθεί ο κώδικας **SemanticSegmentation.py**, στον οποίον επισημαίνονται πέντε διαφορετικά χαρακτηριστικά εικονοστοιχείων και τρεις διαφορετικές τεχνικές ταξινόμησης. Το dataset, στο οποίο εφαρμόζεται ο παραπάνω κώδικας, βρίσκεται στο φάκελο Ερώτημα2 και περιέχει δύο RGB εικόνες και τις αντίστοιχες annotated masks τους που φαίνονται παρακάτω:



Τέλος, στο φάκελο Plots, υπάρχουν γραφικές παραστάσεις για τα χαρακτηριστικά των εικόνων στο φάκελο Features και τις predicted annotated masks για κάθε διαφορετικό συνδυασμό τιμές χαρακτηριστικών εισόδου και τεχνικών ταξινόμησης στο φάκελο Predicted_annotated.

3.1. Features

Στο κώδικα **SemanticSegmentation.py**, υλοποιούνται τα εξής χαρακτηριστικά: χρώμα (color), ακμές (edges), εντροπία (entropy), γωνίες (corners) και εξισορρόπηση (equalization). Στις παρακάτω υποενότητες, θα αναφερθούν σημαντικές ιδιότητες αυτών των χαρακτηριστικών.

3.1.1. Χρώμα

Το χρώμα απλουστεύει κατά πολύ την διαδικασία αναγνώρισης αντικειμένων και την εξαγωγή περιοχών. Μεγάλος αριθμός διαφορετικών αποχρώσεων και εντάσεων χρωμάτων μπορούν να διακριθούν εύκολα με το ανθρώπινο μάτι. Παρακάτω, έχουμε τις εντολές για να βρεθούν οι RGB values των δύο εικόνων στο διάστημα [0,1].

```
colorFeatures1 = inputImg1/255. #normalize color values of inputImg1 in [0,1]
colorFeatures2 = inputImg2/255. #normalize color values of inputImg2 in [0,1]
```

3.1.2. Ακμές

Για την ανίχνευση ακμών σε μία εικόνα, θα χρησιμοποιηθεί ο αλγόριθμος Canny. Ο αλγόριθμος Canny αφαιρεί το θόρυβο με ένα Gaussian φίλτρο και βρίσκει το βαθμό και τη κατεύθυνση της ακμής για κάθε εικονοστοιχείο. Στη συνέχεια, αφαιρεί τα εικονοστοιχεία που δεν σχηματίζουν ακμή με βάση την ύπαρξη τοπικού μεγίστου στη γειτονιά του εικονοστοιχείου προς τη κατεύθυνση της κλίσης. Μετά, αποφασίζει ποιες ακμές θα κρατήσει ανάμεσα κάποιας ελάχιστης και μέγιστης τιμής. Παρακάτω, έχουμε τις εντολές για τη παραπάνω διαδικασία στις δύο εικόνες.

```
edgeBasedFeatures1 = cv2.Canny(inputImg1, 10, 10, edges=None, apertureSize=5)/255
edgeBasedFeatures2 = cv2.Canny(inputImg2, 10, 10, edges=None, apertureSize=5)/255
```

3.1.3. Εντροπία

Η εντροπία σχετίζεται με τη πολυπλοκότητα που περιέχεται σε μία γνωστή γειτονιά, η οποία συνήθως ορίζεται από ένα δομικό στοιχείο. Το φίλτρο εντροπίας μπορεί να ανιχνεύσει λεπτές μεταβολές στην τοπική κατανομή του επιπέδου του γκρι. Παρακάτω, έχουμε τις εντολές για τον υπολογισμό της εντροπίας στις δύο εικόνες.

```
grayImg1 = cv2.cvtColor(inputImg1, cv2.COLOR_BGR2GRAY)
grayImg2 = cv2.cvtColor(inputImg2, cv2.COLOR_BGR2GRAY)
entropyBasedFeatures1 = entropy(grayImg1, disk(5))/6. #entropy features of grayImg1
entropyBasedFeatures2 = entropy(grayImg2, disk(5))/6. #entropy features of grayImg2
```

3.1.4. Εξισορρόπηση

Η εξισορρόπηση μίας εικόνας γίνεται με χρήση ιστογράμματος. Παρακάτω, έχουμε τις εντολές για την εξισορρόπηση εικόνων.

```
grayImg1 = cv2.cvtColor(inputImg1, cv2.COLOR_BGR2GRAY)
grayImg2 = cv2.cvtColor(inputImg2, cv2.COLOR_BGR2GRAY)
equalizationBasedFeatures1 = equalize(grayImg1, disk(5))/255 #equalization features of grayImg1
equalizationBasedFeatures2 = equalize(grayImg2, disk(5))/255 #equalization features of grayImg2
```

3.1.5. Γωνίες

Ο Harris Detector βρίσκει τη διαφορά στην ένταση για μια μετατόπιση (u,v) προς όλες τις κατευθύνσεις. Αυτό εκφράζεται ως εξής:

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u,y+v) - I(x,y)]^2$$

Η συνάρτηση παραθύρου w(x,y) μπορεί να είναι ένα ορθογώνιο παράθυρο ή ένα Gaussian παράθυρο που δίνει βάρη στα εικονοστοιχεία που βρίσκονται από κάτω. Παρακάτω, έχουμε τις εντολές για τη ανίχνευση γωνιών στις δύο εικόνες.

```
cornerFeatures1=cv2.cornerHarris(grayImg1, 8, 3 , 0.04)/255 #corner features of grayImg1
cornerFeatures1= cv2.dilate(cornerFeatures1,None)# Applying dilation to increase the object area and to emphasize features
cornerFeatures2=cv2.cornerHarris(grayImg2, 8, 3 , 0.04)/255 #corner features of grayImg2
cornerFeatures2= cv2.dilate(cornerFeatures2,None)# Applying dilation to increase the object area and to emphasize features
```

3.2. Παρουσίαση και σχολιασμός αποτελεσμάτων

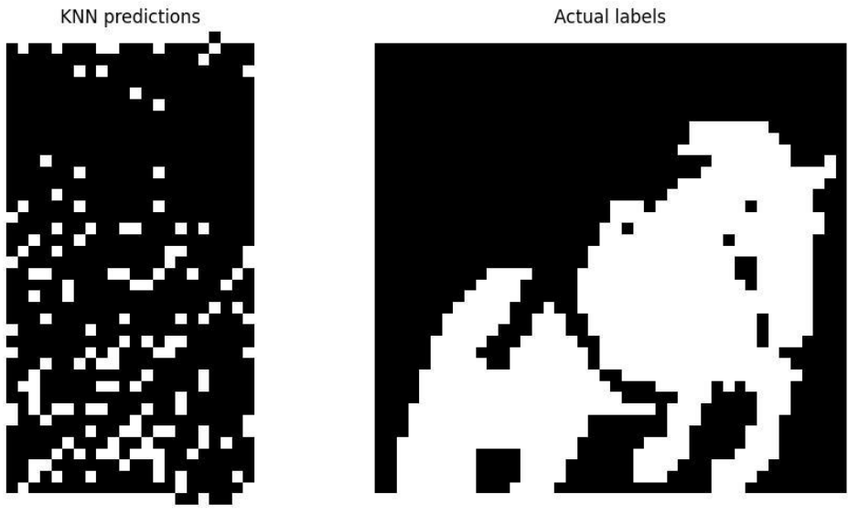
Στο κώδικα **SemanticSegmentation.py**, υλοποιήθηκαν για τον υπολογισμό των accuracy, precision, recall και f1 scores οι εξής τεχνικές ταξινόμησης: KNeighborsClassifier, DecisionTreeClassifier και Gaussian Naïve Bayes classifier. Στο φύλλο **SemanticSegmentation** του αρχείου **results.xlsx**, έχουμε τα αποτελέσματα σε κάθε εικόνα για διαφορετικές τιμές χαρακτηριστικών εισόδου σε εικονοστοιχεία και τεχνικές ταξινόμησης.

Input features type	Image ID	Classifier Used	Accuracy (tr)	Precision (tr)	Recall (tr)	F1 score (tr)	Accuracy (te)	Precision (te)	Recall (te)	F1 score (te)
color	horse.jpg	KNeighborsClassifier	0.75	0.69	0.62	0.63	0.74	0.69	0.64	0.65
color	horse.jpg	DecisionTreeClassifier	1.00	1.00	1.00	1.00	0.65	0.58	0.59	0.58
color	horse.jpg	GaussianNB	0.69	0.59	0.56	0.56	0.74	0.68	0.64	0.65
color	sailboat.jpg	KNeighborsClassifier	0.85	0.76	0.57	0.58	0.83	0.61	0.52	0.52
color	sailboat.jpg	DecisionTreeClassifier	0.99	1.00	0.98	0.99	0.75	0.54	0.54	0.54
color	sailboat.jpg	GaussianNB	0.8	0.50	0.50	0.49	0.8	0.53	0.52	0.51
edge,entropy,equalization,corner	horse.jpg	KNeighborsClassifier	0.74	0.74	0.69	0.7	0.63	0.57	0.56	0.55
edge,entropy,equalization,corner	horse.jpg	DecisionTreeClassifier	1.00	1.00	1.00	1.00	0.60	0.57	0.57	0.57
edge,entropy,equalization,corner	horse.jpg	GaussianNB	0.62	0.31	0.50	0.38	0.65	0.33	0.50	0.40
edge,entropy,equalization,corner	sailboat.jpg	KNeighborsClassifier	0.9	0.76	0.57	0.59	0.88	0.51	0.50	0.48
edge,entropy,equalization,corner	sailboat.jpg	DecisionTreeClassifier	1.00	1.00	1.00	1.00	0.81	0.54	0.55	0.54
edge,entropy,equalization,corner	sailboat.jpg	GaussianNB	0.89	0.45	0.50	0.47	0.89	0.45	0.50	0.47
color,edge,entropy,equalization,corner	horse.jpg	KNeighborsClassifier	0.72	0.72	0.70	0.71	0.56	0.55	0.55	0.55
color,edge,entropy,equalization,corner	horse.jpg	DecisionTreeClassifier	1.00	1.00	1.00	1.00	0.59	0.58	0.58	0.58
color,edge,entropy,equalization,corner	horse.jpg	GaussianNB	0.58	0.57	0.54	0.49	0.56	0.52	0.51	0.46
color,edge,entropy,equalization,corner	sailboat.jpg	KNeighborsClassifier	0.81	0.76	0.73	0.74	0.70	0.63	0.62	0.63
color,edge,entropy,equalization,corner	sailboat.jpg	DecisionTreeClassifier	1.00	1.00	1.00	1.00	0.69	0.62	0.63	0.63
color,edge,entropy,equalization,corner	sailboat.jpg	GaussianNB	0.72	0.36	0.50	0.42	0.72	0.36	0.50	0.42

Στα παραπάνω αποτελέσματα, παρατηρούμε ότι η καλύτερη τεχνική ταξινόμησης για τις τιμές των pixel αυτών των δύο εικόνων είναι ο DecisionTreeClassifier. Όσο αφορά για τα χαρακτηριστικά, το χρώμα δίνει τα καλύτερα αποτελέσματα για την εικόνα “horse.jpg”, ενώ όλα τα υπόλοιπα χαρακτηριστικά εκτός από το χρώμα δίνουν τα καλύτερα αποτελέσματα για την εικόνα “sailboat.jpg”. Επίσης, τα ποσοστά είναι μέχρι σε κάποιο ικανοποιητικό βαθμό για τη δημιουργία εφαρμογής ή ενός εμπορικού προϊόντος. Αλλά, με ένα μεγαλύτερο dataset ίσως να είχαμε καλύτερα αποτελέσματα.

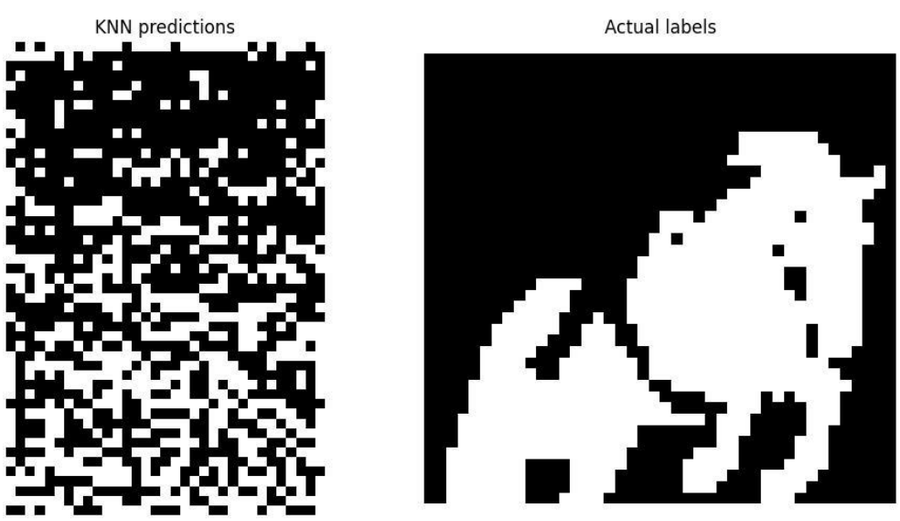
3.3. Γραφικές Παραστάσεις predicted και original annotated masks

Στις επόμενες εικόνες, έχουμε γραφικές παραστάσεις των annotated masks των δύο εικόνων με KNeighborsClassifier για διάφορες τιμές χαρακτηριστικών.

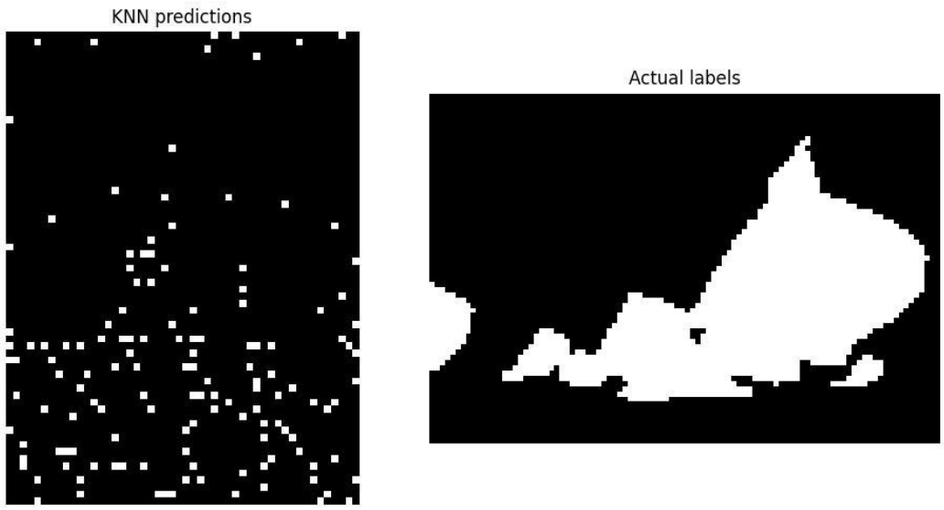


Εικόνα 3.3.1: Γραφικές παραστάσεις με RGB values και KNeighborsClassifier

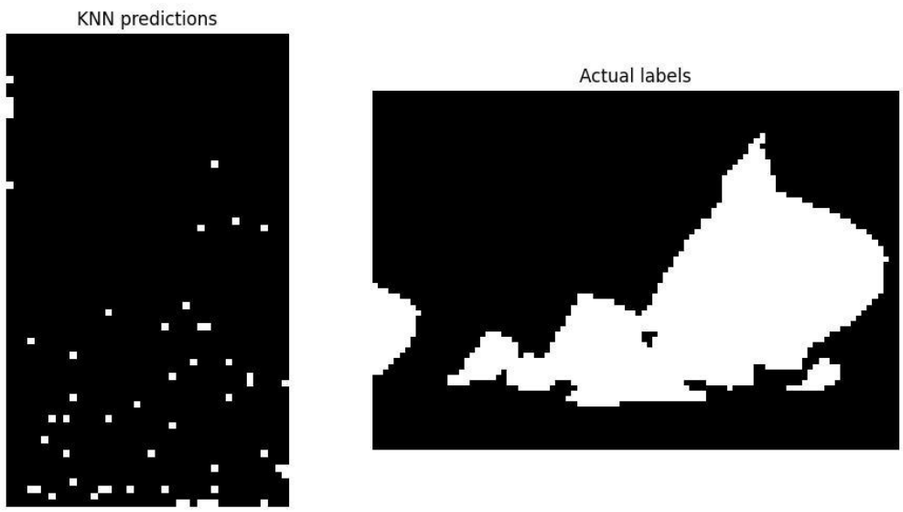
Εικόνα 3.3.2: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και KNeighborsClassifier



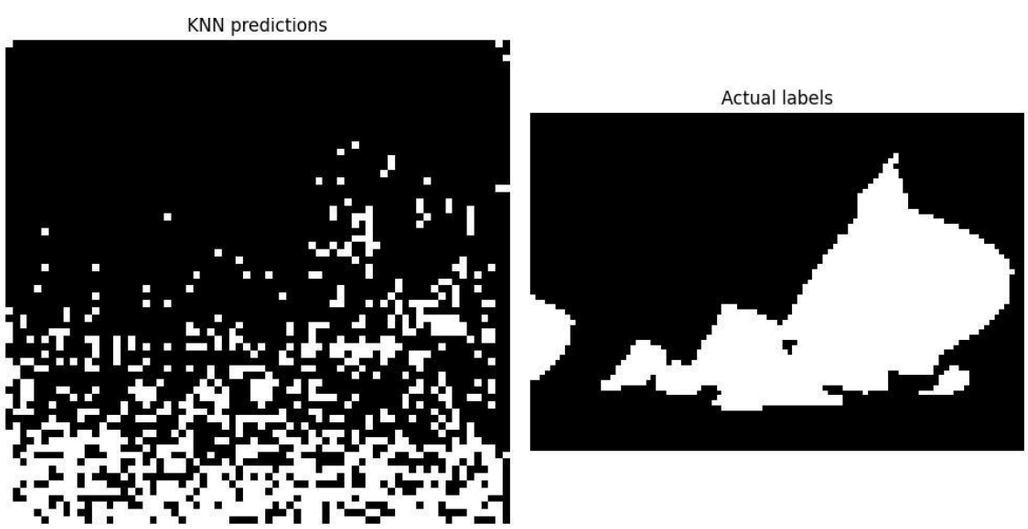
Εικόνα 3.3.3: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών και KNeighborsClassifier



Εικόνα 3.3.4: Γραφικές παραστάσεις με RGB values και KNeighborsClassifier

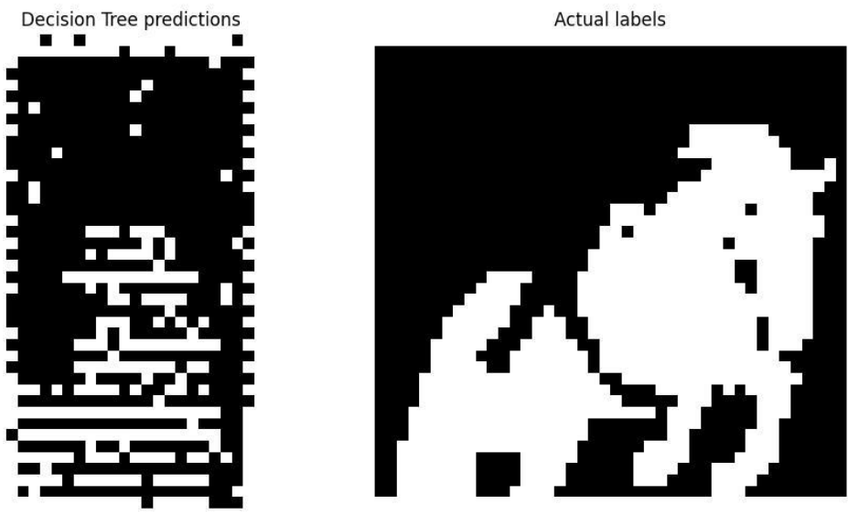


Εικόνα 3.3.5: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και KNeighborsClassifier

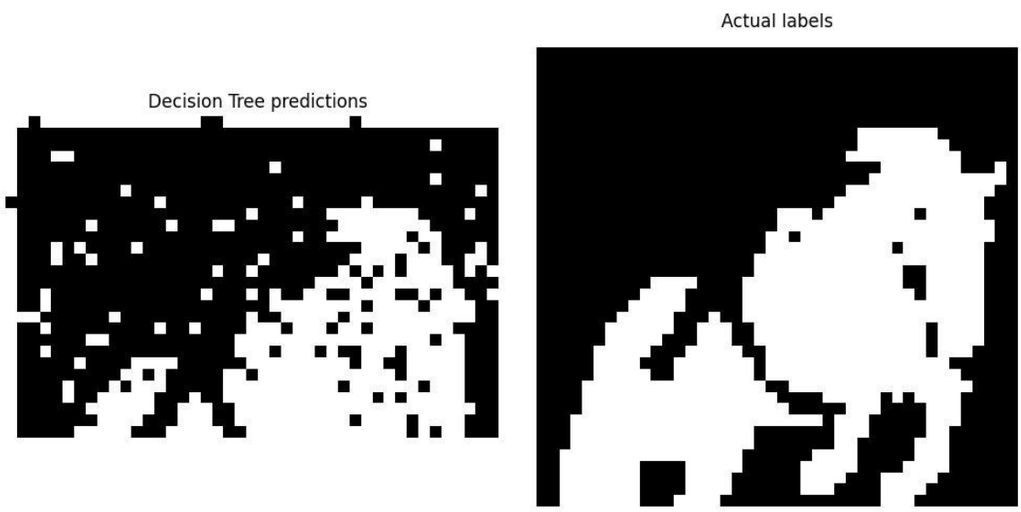


Εικόνα 3.3.6: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών και KNeighborsClassifier

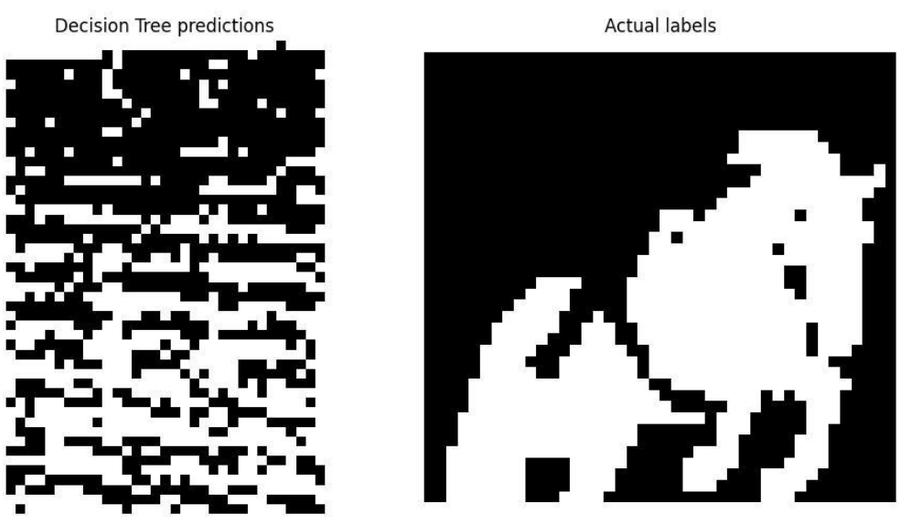
Στις επόμενες εικόνες, έχουμε γραφικές παραστάσεις των annotated masks των δύο εικόνων με DecisionTreeClassifier για διάφορες τιμές χαρακτηριστικών.



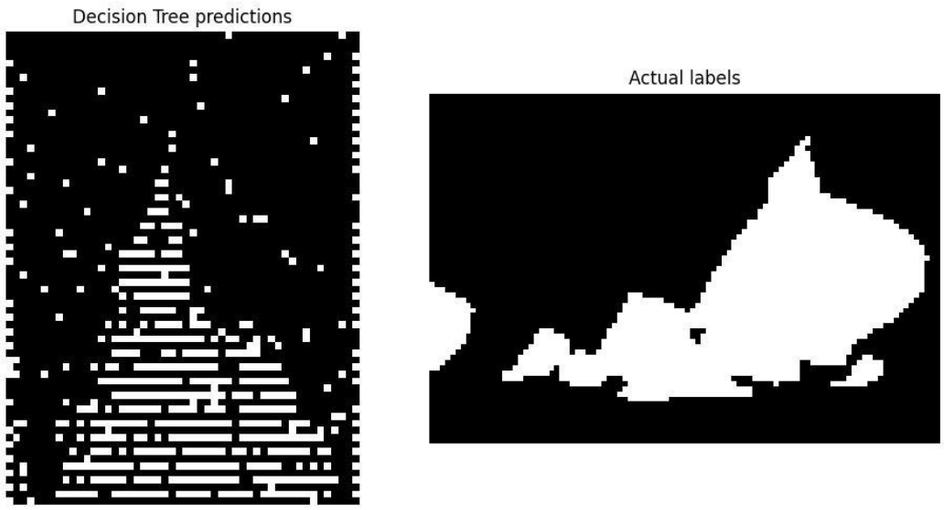
Εικόνα 3.3.7: Γραφικές παραστάσεις με RGB values και DecisionTreeClassifier



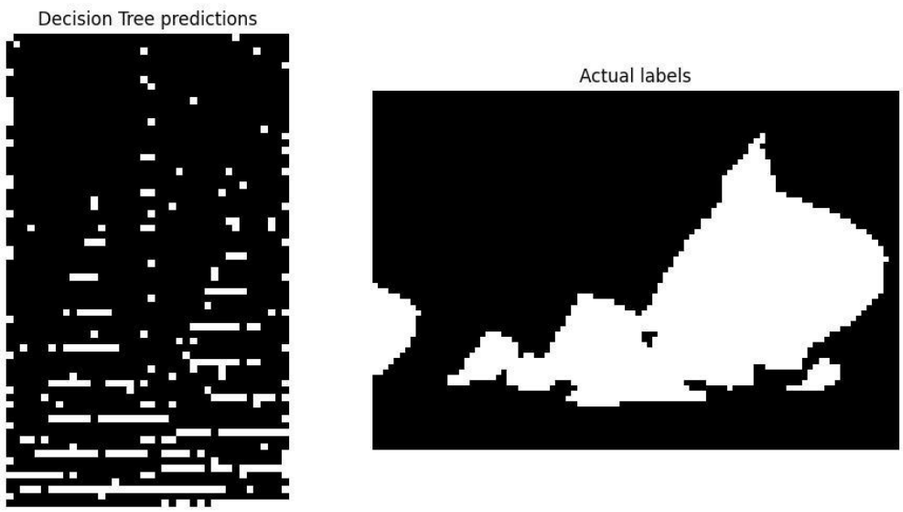
Εικόνα 3.3.8: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και DecisionTreeClassifier



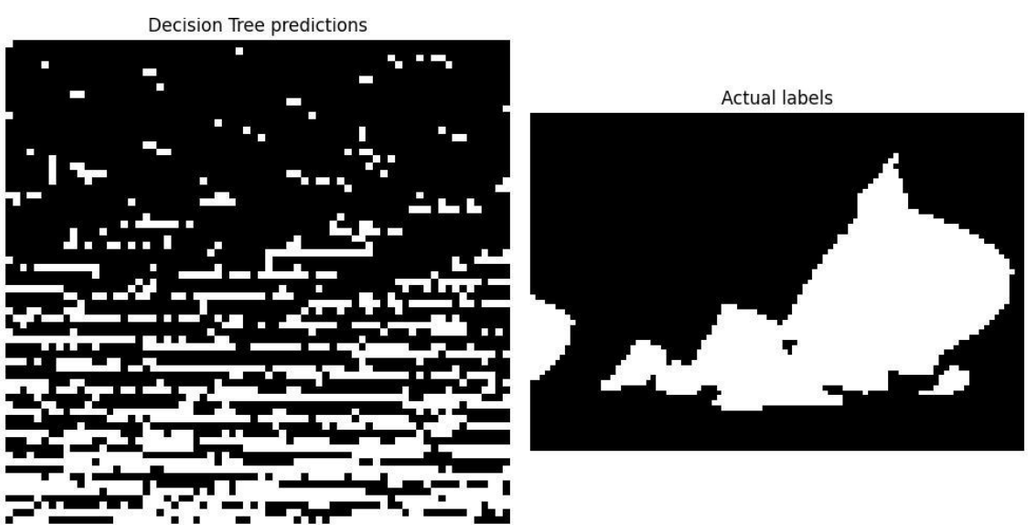
Εικόνα 3.3.9: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών και DecisionTreeClassifier



Εικόνα 3.3.10: Γραφικές παραστάσεις με RGB values και DecisionTreeClassifier

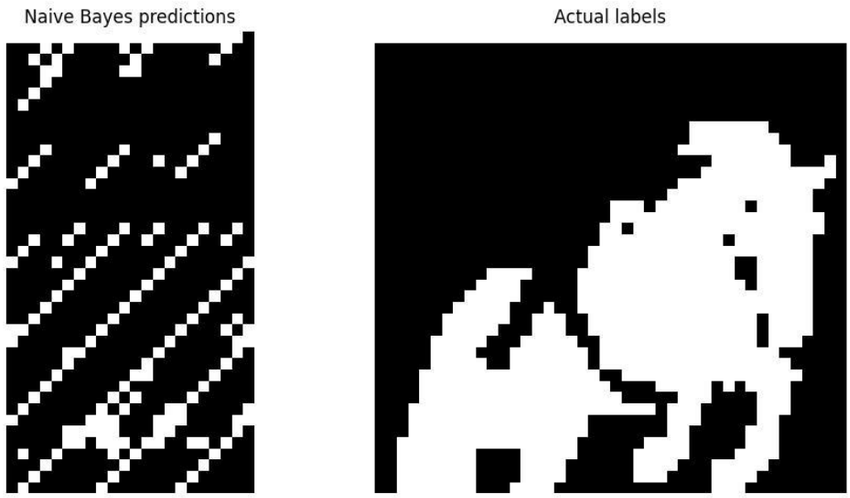


Εικόνα 3.3.11: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και DecisionTreeClassifier

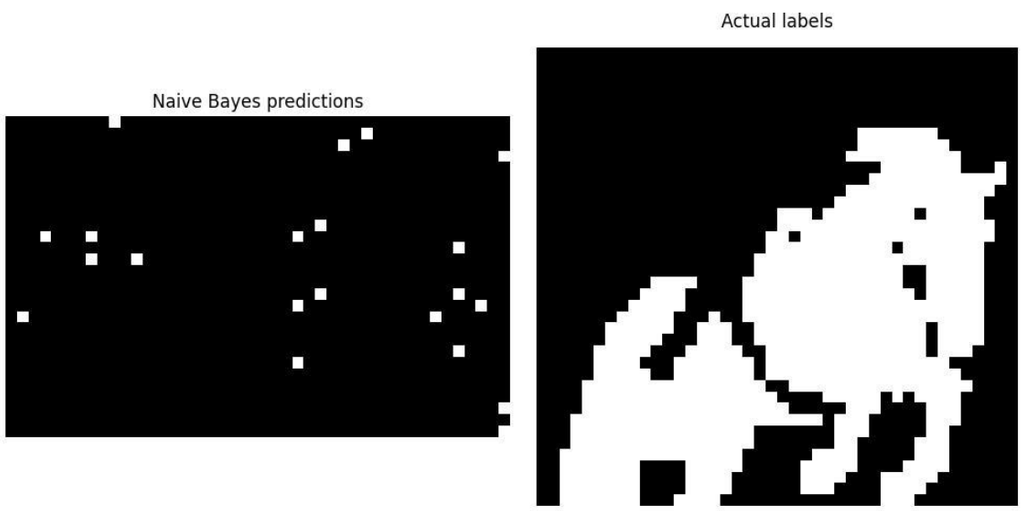


Εικόνα 3.3.12: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και DecisionTreeClassifier

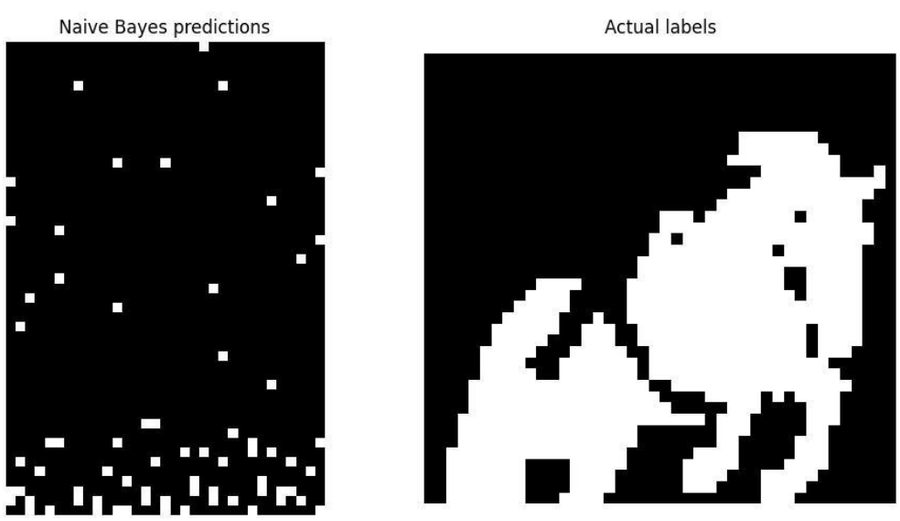
Στις επόμενες εικόνες, έχουμε γραφικές παραστάσεις των annotated masks των δύο εικόνων με Gaussian Naïve Bayes Classifier για διάφορες τιμές χαρακτηριστικών.



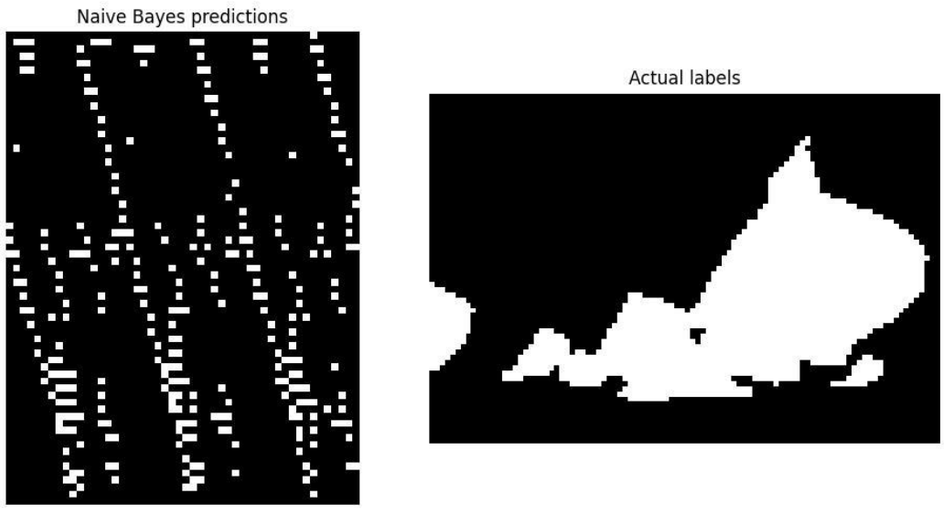
Εικόνα 3.3.13: Γραφικές παραστάσεις με RGB values και Gaussian Naïve Bayes Classifier



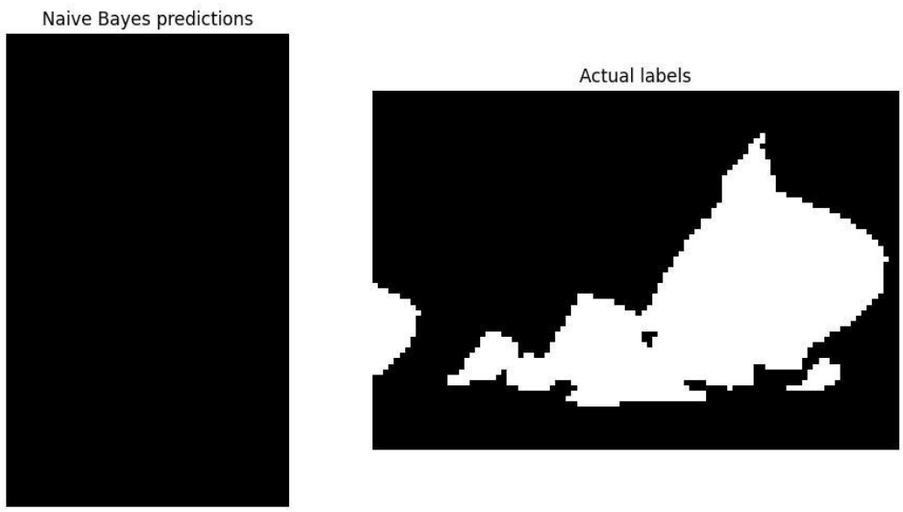
Εικόνα 3.3.14: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και Gaussian Naïve Bayes Classifier



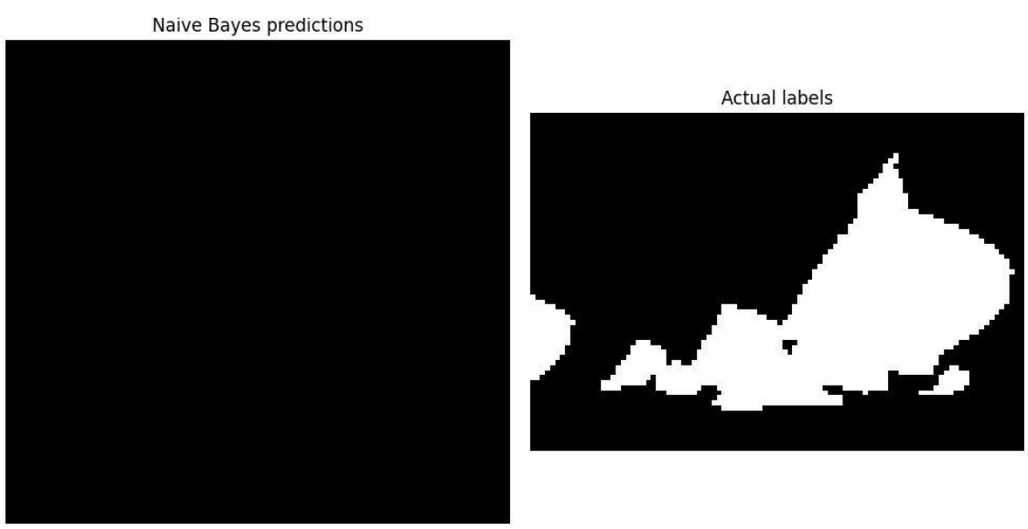
Εικόνα 3.3.15: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών και Gaussian Naïve Bayes Classifier



Εικόνα 3.3.16: Γραφικές παραστάσεις με RGB values και Gaussian Naïve Bayes Classifier



Εικόνα 3.3.17: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών εκτός από RGB values και Gaussian Naïve Bayes Classifier



Εικόνα 3.3.18: Γραφικές παραστάσεις με όλες τις τιμές χαρακτηριστικών και Gaussian Naïve Bayes Classifier

Παρατηρούμε ότι οι γραφικές παραστάσεις των annotated masks των δύο εικόνων με DecisionTreeClassifier επαληθεύουν τις τιμές επίδοσης με βάση τα performance scores. Αντίθετα, οι γραφικές παραστάσεις των annotated masks των δύο εικόνων με Gaussian Naïve Bayes Classifier και KNeighborsClassifier απορρίπτουν τις τιμές επίδοσης με βάση τα performance scores.

4. Συμπεράσματα

Στη ταξινόμηση εικόνων, κάθε αλγόριθμος εξαγωγής συγκεκριμένων χαρακτηριστικών και συσταδοποίησης δημιουργεί διαφορετικά accuracy, precision, recall και f1 scores. Ο kmeans είναι αποδοτικός σε αναλογία train data/test data 60%/40%, ενώ ο Meanshift σε αναλογία train data/test data 80%/20%. Επίσης, στη σημασιολογική κατάτμηση, ο DecisionTreeClassifier είναι η καλύτερη τεχνική ταξινόμησης σε αντίθεση από τους KNeighborsClassifier και Naïve Bayes Classifier.